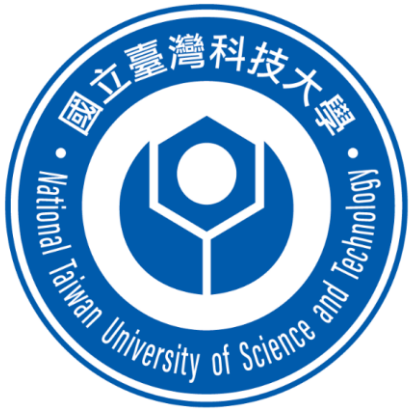
NATIONAL TAIWAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

.



COURSE:

SOFTWARE ENGINEERING IN CONSTRUCTION INFORMATION SYSTEMS

ASSIGNMENT 3

ETL: EXTRACT / TRANSFORM / LOAD

SUPPLIX - Construction Material Supplier Management Platform

INSTRUCTOR: **ASSOCIATE PROFESSOR YO-MING HSIEH**

STUDENT: **DAU THUY DUNG** **D11505807**

HUANG CHING **M11505502**

1. DATABASE AND TABLE CREATION

The SUPPLIX database was developed to support construction material supplier management and procurement activities. The database schema consists of 15 application tables, one historical transaction table, three staging tables, and two ETL rejection tables. Primary keys, foreign keys, unique constraints, check constraints, and triggers are implemented to maintain data integrity and enforce business rules.

1.1. Create database

Create the SUPPLIX database using UTF-8 character encoding and select it as the active database.

```
CREATE DATABASE SUPPLIX_A3_SUBMISSION
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
USE SUPPLIX_A3_SUBMISSION;
```

```
mysql> USE SUPPLIX_A3_SUBMISSION;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW DATABASES;
+----------------------+
| Database             |
+----------------------+
| McDonald             |
| SUPPLIX_A3           |
| SUPPLIX_A3_SUBMISSION |
| information_schema   |
| mysql                |
| performance_schema   |
| sys                  |
+----------------------+
7 rows in set (0.001 sec)

mysql> SELECT DATABASE() AS current_database;
+----------------------+
| current_database     |
+----------------------+
| SUPPLIX_A3_SUBMISSION |
+----------------------+
1 row in set (0.000 sec)
```

1.2. Create application tables

To establish the relational database structure required for the core functions of the SUPPLIX platform, including user management, construction projects, suppliers, materials, quotation requests, purchase orders, and deliveries.

The application database consists of 15 interconnected tables. Each table is designed to store a specific category of information, while primary keys, foreign keys, and other constraints maintain data integrity and consistency.

1.2.1. Create User table

To store user account information and distinguish between General Contractors (GC), Suppliers, and Project Managers (PM).

```
CREATE TABLE `User` (
    user_id VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(200) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('GC','Supplier','PM') NOT NULL,
    phone VARCHAR(100),
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

The CREATE TABLE statement establishes the User table with seven columns. The user_id field serves as the primary key, while the UNIQUE constraint prevents duplicate email addresses. The NOT NULL constraints ensure that essential account information is provided. The role field uses an ENUM data type to restrict users to three predefined roles. The created_at field automatically records the account creation time, and InnoDB supports relational integrity.

```
mysql> SHOW CREATE TABLE User\G
*************************** 1. row ***************************
       Table: User
Create Table: CREATE TABLE `User` (
  `user_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `name` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email` varchar(200) COLLATE utf8mb4_unicode_ci NOT NULL,
  `password_hash` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `role` enum('GC','Supplier','PM') COLLATE utf8mb4_unicode_ci NOT NULL,
  `phone` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`user_id`),
  UNIQUE KEY `email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.2.2. Create Project table

To store construction project information and establish a relationship between each project and its registered owner.

```
CREATE TABLE Project (
   project_id VARCHAR(20) PRIMARY KEY,
   owner_user_id VARCHAR(20) NOT NULL,
   name VARCHAR(200) NOT NULL,
   location VARCHAR(255),
   status ENUM('active','completed') NOT NULL DEFAULT 'active',
   start_date DATE,
   end_date DATE,
   CONSTRAINT fk_project_owner FOREIGN KEY (owner_user_id) REFERENCES `User`(user_id),
   CONSTRAINT chk_project_dates CHECK (start_date IS NULL OR end_date IS NULL OR end_date >= start_date)
) ENGINE=InnoDB;
```

The Project table contains seven columns representing project identification, ownership, location, status, and scheduling information. The project_id column uniquely identifies each project. The owner_user_id column references the User table through a foreign key, preventing projects from being assigned to nonexistent users.

The status field allows only active or completed, with active as the default value. The chk_project_dates constraint ensures that the end date is not earlier than the start date when both dates are specified.

```
mysql> SHOW CREATE TABLE Project\G
*************************** 1. row ***************************
       Table: Project
Create Table: CREATE TABLE `Project` (
  `project_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `owner_user_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `name` varchar(200) COLLATE utf8mb4_unicode_ci NOT NULL,
  `location` varchar(255) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `status` enum('active','completed') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'active',
  `start_date` date DEFAULT NULL,
  `end_date` date DEFAULT NULL,
  PRIMARY KEY (`project_id`),
  KEY `fk_project_owner` (`owner_user_id`),
  CONSTRAINT `fk_project_owner` FOREIGN KEY (`owner_user_id`) REFERENCES `User` (`user_id`),
  CONSTRAINT `chk_project_dates` CHECK (((`start_date` is null) or (`end_date` is null) or (`end_date` >= `start_date`)))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.3. Create ProjectMember table

To record the assignment of project managers to construction projects.

```
CREATE TABLE ProjectMember (
   project_id VARCHAR(20) NOT NULL,
   user_id VARCHAR(20) NOT NULL,
   assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   PRIMARY KEY (project_id, user_id),
   CONSTRAINT fk_pm_project FOREIGN KEY (project_id) REFERENCES Project(project_id),
   CONSTRAINT fk_pm_user FOREIGN KEY (user_id) REFERENCES `User`(user_id)
) ENGINE=InnoDB;
```

The composite primary key (project_id, user_id) prevents duplicate assignments. Both columns reference their respective parent tables, ensuring that every membership belongs to an existing project and user. assigned_at automatically records the assignment time.

```
mysql> SHOW CREATE TABLE ProjectMember\G
*************************** 1. row ***************************
       Table: ProjectMember
Create Table: CREATE TABLE `ProjectMember` (
  `project_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `user_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `assigned_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`project_id`,`user_id`),
  KEY `fk_pm_user` (`user_id`),
  CONSTRAINT `fk_pm_project` FOREIGN KEY (`project_id`) REFERENCES `Project` (`project_id`),
  CONSTRAINT `fk_pm_user` FOREIGN KEY (`user_id`) REFERENCES `User` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.4. Create Supplier table

To maintain supplier company profiles and their optional association with platform user accounts.

```
CREATE TABLE Supplier (
   supplier_id VARCHAR(20) PRIMARY KEY,
   user_id VARCHAR(20) NULL UNIQUE,
   company_name VARCHAR(200) NOT NULL,
   description TEXT,
   location TEXT,
   phone VARCHAR(100),
   email VARCHAR(200), -- imported company contact email; not login email
   is_verified BOOLEAN NOT NULL DEFAULT FALSE,
   created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT fk_supplier_user FOREIGN KEY (user_id) REFERENCES `User`(user_id)
) ENGINE=InnoDB;
```

supplier_id uniquely identifies each company. The optional, unique user_id links a supplier profile to at most one login account. Contact details are stored separately from account credentials. is_verified defaults to FALSE and created_at records the creation time.

```
mysql> SHOW CREATE TABLE Supplier\G
*************************** 1. row ***************************
       Table: Supplier
Create Table: CREATE TABLE `Supplier` (
  `supplier_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `user_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `company_name` varchar(200) COLLATE utf8mb4_unicode_ci NOT NULL,
  `description` text COLLATE utf8mb4_unicode_ci,
  `location` text COLLATE utf8mb4_unicode_ci,
  `phone` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `email` varchar(200) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `is_verified` tinyint(1) NOT NULL DEFAULT '0',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`supplier_id`),
  UNIQUE KEY `user_id` (`user_id`),
  CONSTRAINT `fk_supplier_user` FOREIGN KEY (`user_id`) REFERENCES `User` (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.5. Create Material table

To store shared construction material specifications used throughout procurement.

```
CREATE TABLE Material (
   material_id VARCHAR(20) PRIMARY KEY,
   name TEXT NOT NULL,
   category VARCHAR(100) NULL, -- unavailable in materials.tsv; enrich later
   unit VARCHAR(50) NOT NULL,
   specification TEXT
) ENGINE=InnoDB;
```

material_id uniquely identifies each material, while name and unit are mandatory. category and specification are optional descriptive attributes. The nullable category supports importing materials whose source file does not provide a category.

```
mysql> SHOW CREATE TABLE Material\G
*************************** 1. row ***************************
       Table: Material
Create Table: CREATE TABLE `Material` (
  `material_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `name` text COLLATE utf8mb4_unicode_ci NOT NULL,
  `category` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `unit` varchar(50) COLLATE utf8mb4_unicode_ci NOT NULL,
  `specification` text COLLATE utf8mb4_unicode_ci,
  PRIMARY KEY (`material_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.6. Create SupplierMaterial table

To associate suppliers with materials they offer and record their listing prices and availability.

```
CREATE TABLE SupplierMaterial (
   supplier_material_id BIGINT AUTO_INCREMENT PRIMARY KEY,
   supplier_id VARCHAR(20) NOT NULL,
   material_id VARCHAR(20) NOT NULL,
   unit_price DECIMAL(18,4) NOT NULL,
   stock_available BOOLEAN NOT NULL,
   UNIQUE KEY uq_supplier_material (supplier_id,material_id),
```

```
    CONSTRAINT fk_sm_supplier FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id),
    CONSTRAINT fk_sm_material FOREIGN KEY (material_id) REFERENCES Material(material_id),
    CONSTRAINT chk_sm_price CHECK (unit_price >= 0)
) ENGINE=InnoDB;
```

The auto-incrementing supplier_material_id identifies each listing. The two foreign keys link it to an existing supplier and material. A composite UNIQUE constraint prevents duplicate supplier–material listings, and the CHECK constraint disallows negative unit prices.

```
mysql> SHOW CREATE TABLE SupplierMaterial\G
*************************** 1. row ***************************
       Table: SupplierMaterial
Create Table: CREATE TABLE `SupplierMaterial` (
  `supplier_material_id` bigint NOT NULL AUTO_INCREMENT,
  `supplier_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `material_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `unit_price` decimal(18,4) NOT NULL,
  `stock_available` tinyint(1) NOT NULL,
  PRIMARY KEY (`supplier_material_id`),
  UNIQUE KEY `uq_supplier_material` (`supplier_id`,`material_id`),
  KEY `fk_sm_material` (`material_id`),
  CONSTRAINT `fk_sm_material` FOREIGN KEY (`material_id`) REFERENCES `Material` (`material_id`),
  CONSTRAINT `fk_sm_supplier` FOREIGN KEY (`supplier_id`) REFERENCES `Supplier` (`supplier_id`),
  CONSTRAINT `chk_sm_price` CHECK ((`unit_price` >= 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.7. Create RFQ table

To register requests for quotations associated with construction projects.

```
CREATE TABLE RFQ (
    rfq_id VARCHAR(20) PRIMARY KEY,
    project_id VARCHAR(20) NOT NULL,
    created_by VARCHAR(20) NOT NULL,
    deadline DATE NOT NULL,
    status ENUM('draft','open','closed') NOT NULL DEFAULT 'draft',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_rfq_project FOREIGN KEY (project_id) REFERENCES Project(project_id),
    CONSTRAINT fk_rfq_creator FOREIGN KEY (created_by) REFERENCES `User`(user_id)
```

```
) ENGINE=InnoDB;
```

rfq_id identifies each request. Foreign keys link it to an existing project and creator. deadline is required, and status is limited to draft, open, or closed, defaulting to draft. created_at stores the creation timestamp.

```
mysql> SHOW CREATE TABLE RFQ\G
*************************** 1. row ***************************
       Table: RFQ
Create Table: CREATE TABLE `RFQ` (
  `rfq_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `project_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `created_by` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `deadline` date NOT NULL,
  `status` enum('draft','open','closed') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'draft',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`rfq_id`),
  KEY `fk_rfq_project` (`project_id`),
  KEY `fk_rfq_creator` (`created_by`),
  CONSTRAINT `fk_rfq_creator` FOREIGN KEY (`created_by`) REFERENCES `User` (`user_id`),
  CONSTRAINT `fk_rfq_project` FOREIGN KEY (`project_id`) REFERENCES `Project` (`project_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.003 sec)
```

1.2.8. Create RFQ_Supplier table

To record which suppliers are invited to respond to each request for quotation.

```
CREATE TABLE RFQ_Supplier (
   rfq_id VARCHAR(20) NOT NULL,
   supplier_id VARCHAR(20) NOT NULL,
   status ENUM('invited','responded','declined') NOT NULL DEFAULT 'invited',
   invited_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   PRIMARY KEY (rfq_id,supplier_id),
   CONSTRAINT fk_rs_rfq FOREIGN KEY (rfq_id) REFERENCES RFQ(rfq_id),
   CONSTRAINT fk_rs_supplier FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
) ENGINE=InnoDB;
```

The composite primary key (rfq_id, supplier_id) ensures that each supplier is invited at most once per RFQ. Foreign keys validate the referenced RFQ and supplier. The invitation status defaults to invited, and invited_at records when the invitation was created.

```
mysql> SHOW CREATE TABLE RFQ_Supplier\G
*************************** 1. row ***************************
       Table: RFQ_Supplier
Create Table: CREATE TABLE `RFQ_Supplier` (
  `rfq_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `supplier_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `status` enum('invited','responded','declined') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'invited',
  `invited_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`rfq_id`,`supplier_id`),
  KEY `fk_rs_supplier` (`supplier_id`),
  CONSTRAINT `fk_rs_rfq` FOREIGN KEY (`rfq_id`) REFERENCES `RFQ` (`rfq_id`),
  CONSTRAINT `fk_rs_supplier` FOREIGN KEY (`supplier_id`) REFERENCES `Supplier` (`supplier_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.2.9. Create RFQ_Item table

To specify the materials, quantities, and required dates requested within each RFQ.

```
CREATE TABLE RFQ_Item (
    rfq_item_id VARCHAR(20) PRIMARY KEY,
    rfq_id VARCHAR(20) NOT NULL,
    material_id VARCHAR(20) NOT NULL,
    quantity DECIMAL(18,4) NOT NULL,
    required_date DATE,
    CONSTRAINT fk_ri_rfq FOREIGN KEY (rfq_id) REFERENCES RFQ(rfq_id),
    CONSTRAINT fk_ri_material FOREIGN KEY (material_id) REFERENCES Material(material_id),
    CONSTRAINT chk_ri_quantity CHECK (quantity > 0)
) ENGINE=InnoDB;
```

Each rfq_item_id uniquely identifies a requested line. Foreign keys connect the line to its RFQ and shared Material record. The quantity must be greater than zero, while required_date may be omitted.

```
mysql> SHOW CREATE TABLE RFQ_Item\G
*************************** 1. row ***************************
       Table: RFQ_Item
Create Table: CREATE TABLE `RFQ_Item` (
  `rfq_item_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `rfq_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `material_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quantity` decimal(18,4) NOT NULL,
  `required_date` date DEFAULT NULL,
  PRIMARY KEY (`rfq_item_id`),
  KEY `fk_ri_rfq` (`rfq_id`),
  KEY `fk_ri_material` (`material_id`),
  CONSTRAINT `fk_ri_material` FOREIGN KEY (`material_id`) REFERENCES `Material` (`material_id`),
  CONSTRAINT `fk_ri_rfq` FOREIGN KEY (`rfq_id`) REFERENCES `RFQ` (`rfq_id`),
  CONSTRAINT `chk_ri_quantity` CHECK ((`quantity` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.10. Create Quotation table

To store supplier quotations submitted in response to RFQs.

```
CREATE TABLE Quotation (
    quotation_id VARCHAR(20) PRIMARY KEY,
    rfq_id VARCHAR(20) NOT NULL,
    supplier_id VARCHAR(20) NOT NULL,
    delivery_terms TEXT,
    status ENUM('pending','accepted','rejected') NOT NULL DEFAULT 'pending',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_quotation_rfq_supplier (rfq_id,supplier_id),
    -- MySQL permits multiple NULLs: only one accepted quotation per RFQ.
    accepted_rfq_id VARCHAR(20) GENERATED ALWAYS AS (CASE WHEN status = 'accepted' THEN rfq_id ELSE NULL END) STORED,
    UNIQUE KEY uq_one_accepted_quotation (accepted_rfq_id),
    CONSTRAINT fk_quotation_invitation FOREIGN KEY (rfq_id,supplier_id) REFERENCES RFQ_Supplier(rfq_id,supplier_id)
) ENGINE=InnoDB;
```

The composite foreign key (rfq_id, supplier_id) requires a matching RFQ invitation. The UNIQUE constraint allows one quotation per invited supplier per RFQ. A generated accepted_rfq_id and its UNIQUE key restrict each RFQ to at most one accepted quotation; nonaccepted quotations yield NULL.

```
mysql> SHOW CREATE TABLE Quotation\G
*************************** 1. row ***************************
       Table: Quotation
Create Table: CREATE TABLE `Quotation` (
  `quotation_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `rfq_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `supplier_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `delivery_terms` text COLLATE utf8mb4_unicode_ci,
  `status` enum('pending','accepted','rejected') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'pending',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `accepted_rfq_id` varchar(20) COLLATE utf8mb4_unicode_ci GENERATED ALWAYS AS ((case when (`status` = _utf8mb4'accepted') then `rfq_
id` else NULL end)) STORED,
  PRIMARY KEY (`quotation_id`),
  UNIQUE KEY `uq_quotation_rfq_supplier` (`rfq_id`,`supplier_id`),
  UNIQUE KEY `uq_one_accepted_quotation` (`accepted_rfq_id`),
  CONSTRAINT `fk_quotation_invitation` FOREIGN KEY (`rfq_id`, `supplier_id`) REFERENCES `RFQ_Supplier` (`rfq_id`, `supplier_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.2.11. Create Quotation_Item table

To store material-level offers, quantities, prices, and proposed delivery dates within supplier quotations.

```
CREATE TABLE Quotation_Item (
   quotation_item_id VARCHAR(20) PRIMARY KEY,
   quotation_id VARCHAR(20) NOT NULL,
   rfq_item_id VARCHAR(20) NOT NULL,
   unit_price DECIMAL(18,4) NOT NULL,
   quantity DECIMAL(18,4) NOT NULL,
   delivery_date DATE NOT NULL,
   UNIQUE KEY uq_qi_quotation_item (quotation_id,rfq_item_id),
   CONSTRAINT fk_qi_quotation FOREIGN KEY (quotation_id) REFERENCES Quotation(quotation_id),
   CONSTRAINT fk_qi_rfq_item FOREIGN KEY (rfq_item_id) REFERENCES RFQ_Item(rfq_item_id),
   CONSTRAINT chk_qi_price CHECK (unit_price >= 0),
   CONSTRAINT chk_qi_quantity CHECK (quantity > 0)
) ENGINE=InnoDB;
```

Foreign keys link each line to an existing quotation and RFQ item. The composite UNIQUE key prevents quoting the same RFQ item twice within one quotation. CHECK constraints require positive quantities and nonnegative unit prices, and delivery_date is mandatory.

```
mysql> SHOW CREATE TABLE Quotation_Item\G
*************************** 1. row ***************************
       Table: Quotation_Item
Create Table: CREATE TABLE `Quotation_Item` (
  `quotation_item_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quotation_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `rfq_item_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `unit_price` decimal(18,4) NOT NULL,
  `quantity` decimal(18,4) NOT NULL,
  `delivery_date` date NOT NULL,
  PRIMARY KEY (`quotation_item_id`),
  UNIQUE KEY `uq_qi_quotation_item` (`quotation_id`,`rfq_item_id`),
  KEY `fk_qi_rfq_item` (`rfq_item_id`),
  CONSTRAINT `fk_qi_quotation` FOREIGN KEY (`quotation_id`) REFERENCES `Quotation` (`quotation_id`),
  CONSTRAINT `fk_qi_rfq_item` FOREIGN KEY (`rfq_item_id`) REFERENCES `RFQ_Item` (`rfq_item_id`),
  CONSTRAINT `chk_qi_price` CHECK ((`unit_price` >= 0)),
  CONSTRAINT `chk_qi_quantity` CHECK ((`quantity` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.2.12. Create PurchaseOrder table

To create purchase orders based on supplier quotations.

```
CREATE TABLE PurchaseOrder (
    po_id VARCHAR(20) PRIMARY KEY,
    quotation_id VARCHAR(20) NOT NULL UNIQUE,
    created_by VARCHAR(20) NOT NULL,
    status ENUM('draft','pending','accepted','rejected','in_progress','fulfilled') NOT NULL DEFAULT 'draft',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_po_quotation FOREIGN KEY (quotation_id) REFERENCES Quotation(quotation_id),
    CONSTRAINT fk_po_creator FOREIGN KEY (created_by) REFERENCES `User`(user_id)
) ENGINE=InnoDB;
```

Each purchase order has a unique po_id and references a quotation and its creating user. The UNIQUE quotation_id prevents multiple purchase orders from referring to the same quotation. Status values represent the order lifecycle, starting with draft.

```
mysql> SHOW CREATE TABLE PurchaseOrder\G
*************************** 1. row ***************************
       Table: PurchaseOrder
Create Table: CREATE TABLE `PurchaseOrder` (
  `po_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quotation_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `created_by` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `status` enum('draft','pending','accepted','rejected','in_progress','fulfilled') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'draft
',
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`po_id`),
  UNIQUE KEY `quotation_id` (`quotation_id`),
  KEY `fk_po_creator` (`created_by`),
  CONSTRAINT `fk_po_creator` FOREIGN KEY (`created_by`) REFERENCES `User` (`user_id`),
  CONSTRAINT `fk_po_quotation` FOREIGN KEY (`quotation_id`) REFERENCES `Quotation` (`quotation_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.001 sec)
```

1.2.13. Create PO_Item table

To record the individual order lines derived from quoted items.

```
CREATE TABLE PO_Item (
   po_item_id VARCHAR(20) PRIMARY KEY,
   po_id VARCHAR(20) NOT NULL,
   quotation_item_id VARCHAR(20) NOT NULL,
   quantity DECIMAL(18,4) NOT NULL,
   agreed_unit_price DECIMAL(18,4) NOT NULL,
   UNIQUE KEY uq_po_quotation_item (po_id,quotation_item_id),
   CONSTRAINT fk_poi_po FOREIGN KEY (po_id) REFERENCES PurchaseOrder(po_id),
   CONSTRAINT fk_poi_quotation_item FOREIGN KEY (quotation_item_id) REFERENCES Quotation_Item(quotation_item_id),
   CONSTRAINT chk_poi_quantity CHECK (quantity > 0),
   CONSTRAINT chk_poi_price CHECK (agreed_unit_price >= 0)
) ENGINE=InnoDB;
```

Each line references an existing purchase order and quotation item. The composite UNIQUE key prevents duplicate quotation items within an order. The CHECK constraints enforce positive ordered quantities and nonnegative agreed unit prices.

```
mysql> SHOW CREATE TABLE PO_Item\G
*************************** 1. row ***************************
       Table: PO_Item
Create Table: CREATE TABLE `PO_Item` (
  `po_item_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `po_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quotation_item_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quantity` decimal(18,4) NOT NULL,
  `agreed_unit_price` decimal(18,4) NOT NULL,
  PRIMARY KEY (`po_item_id`),
  UNIQUE KEY `uq_po_quotation_item` (`po_id`,`quotation_item_id`),
  KEY `fk_poi_quotation_item` (`quotation_item_id`),
  CONSTRAINT `fk_poi_po` FOREIGN KEY (`po_id`) REFERENCES `PurchaseOrder` (`po_id`),
  CONSTRAINT `fk_poi_quotation_item` FOREIGN KEY (`quotation_item_id`) REFERENCES `Quotation_Item` (`quotation_item_id`),
  CONSTRAINT `chk_poi_price` CHECK ((`agreed_unit_price` >= 0)),
  CONSTRAINT `chk_poi_quantity` CHECK ((`quantity` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.2.14. Create Delivery table

To track fulfillment and delivery information for purchase orders.

```
CREATE TABLE Delivery (
    delivery_id VARCHAR(20) PRIMARY KEY,
    po_id VARCHAR(20) NOT NULL UNIQUE,
    status ENUM('dispatched','out_for_delivery','delivered','delayed') NOT NULL DEFAULT 'dispatched',
    estimated_date DATE NOT NULL,
    actual_date DATE,
    delay_reason TEXT,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_delivery_po FOREIGN KEY (po_id) REFERENCES PurchaseOrder(po_id)
) ENGINE=InnoDB;
```

Each delivery references one purchase order; the UNIQUE po_id permits at most one delivery record per order. The status field tracks dispatch, transit, delivery, or delay. estimated_date is required, while actual_date and delay_reason are optional. updated_at refreshes when the row changes.

```
mysql> SHOW CREATE TABLE Delivery\G
*************************** 1. row ***************************
       Table: Delivery
Create Table: CREATE TABLE `Delivery` (
  `delivery_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `po_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `status` enum('dispatched','out_for_delivery','delivered','delayed') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'dispatched',
  `estimated_date` date NOT NULL,
  `actual_date` date DEFAULT NULL,
  `delay_reason` text COLLATE utf8mb4_unicode_ci,
  `updated_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`delivery_id`),
  UNIQUE KEY `po_id` (`po_id`),
  CONSTRAINT `fk_delivery_po` FOREIGN KEY (`po_id`) REFERENCES `PurchaseOrder` (`po_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.2.15. Create Document table

To store metadata for documents associated with projects, suppliers, or purchase orders.

```
CREATE TABLE Document (
    document_id VARCHAR(20) PRIMARY KEY,
    project_id VARCHAR(20),
    supplier_id VARCHAR(20),
    po_id VARCHAR(20),
    uploaded_by VARCHAR(20) NOT NULL,
    document_type ENUM('certificate','invoice','safety_sheet','contract') NOT NULL,
    file_name VARCHAR(255) NOT NULL,
    file_url VARCHAR(1000) NOT NULL,
    expiry_date DATE,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_doc_project FOREIGN KEY (project_id) REFERENCES Project(project_id),
    CONSTRAINT fk_doc_supplier FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id),
    CONSTRAINT fk_doc_po FOREIGN KEY (po_id) REFERENCES PurchaseOrder(po_id),
    CONSTRAINT fk_doc_user FOREIGN KEY (uploaded_by) REFERENCES `User`(user_id),
    CONSTRAINT chk_doc_link CHECK (project_id IS NOT NULL OR supplier_id IS NOT NULL OR po_id IS NOT NULL)
) ENGINE=InnoDB;
```

Foreign keys connect documents to the relevant business records and uploader. The document_type ENUM limits supported categories. file_name and file_url are required. The chk_doc_link constraint requires at least one project, supplier, or purchase order association.

```
mysql> SHOW CREATE TABLE Document\G
*************************** 1. row ***************************
       Table: Document
Create Table: CREATE TABLE `Document` (
  `document_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `project_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `supplier_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `po_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `uploaded_by` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `document_type` enum('certificate','invoice','safety_sheet','contract') COLLATE utf8mb4_unicode_ci NOT NULL,
  `file_name` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `file_url` varchar(1000) COLLATE utf8mb4_unicode_ci NOT NULL,
  `expiry_date` date DEFAULT NULL,
  `created_at` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`document_id`),
  KEY `fk_doc_project` (`project_id`),
  KEY `fk_doc_supplier` (`supplier_id`),
  KEY `fk_doc_po` (`po_id`),
  KEY `fk_doc_user` (`uploaded_by`),
  CONSTRAINT `fk_doc_po` FOREIGN KEY (`po_id`) REFERENCES `PurchaseOrder` (`po_id`),
  CONSTRAINT `fk_doc_project` FOREIGN KEY (`project_id`) REFERENCES `Project` (`project_id`),
  CONSTRAINT `fk_doc_supplier` FOREIGN KEY (`supplier_id`) REFERENCES `Supplier` (`supplier_id`),
  CONSTRAINT `fk_doc_user` FOREIGN KEY (`uploaded_by`) REFERENCES `User` (`user_id`),
  CONSTRAINT `chk_doc_link` CHECK (((`project_id` is not null) or (`supplier_id` is not null) or (`po_id` is not null)))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.3. Create business rule triggers

To enforce business rules that cannot be fully implemented using primary keys, foreign keys, and CHECK constraints.

A total of 17 triggers were created to validate user roles, project ownership, quotation consistency, purchase order requirements, and transaction-related restrictions. These triggers are executed automatically before INSERT or UPDATE operations to prevent invalid records from entering the database.

```
-- STEP 1B: Database-side business rules not expressible as simple foreign keys.
-- MySQL client understands DELIMITER when executed with SOURCE / redirected input.
DELIMITER $$
CREATE TRIGGER trg_project_gc_before_insert BEFORE INSERT ON Project FOR EACH ROW
BEGIN
   IF (SELECT role FROM `User` WHERE user_id=NEW.owner_user_id) <> 'GC' THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Project owner must have GC role';
   END IF;
END$$
```

```
CREATE TRIGGER trg_project_gc_before_update BEFORE UPDATE ON Project FOR EACH ROW
BEGIN
  IF (SELECT role FROM `User` WHERE user_id=NEW.owner_user_id) <> 'GC' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Project owner must have GC role';
  END IF;
END$$
CREATE TRIGGER trg_supplier_role_before_insert BEFORE INSERT ON Supplier FOR EACH ROW
BEGIN
  IF NEW.user_id IS NOT NULL AND
    (SELECT role FROM `User` WHERE user_id=NEW.user_id) <> 'Supplier' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Linked supplier account must have Supplier role';
  END IF;
END$$
CREATE TRIGGER trg_supplier_role_before_update BEFORE UPDATE ON Supplier FOR EACH ROW
BEGIN
  IF NEW.user_id IS NOT NULL AND
    (SELECT role FROM `User` WHERE user_id=NEW.user_id) <> 'Supplier' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Linked supplier account must have Supplier role';
  END IF;
END$$
CREATE TRIGGER trg_qi_same_rfq_before_insert BEFORE INSERT ON Quotation_Item FOR EACH ROW
BEGIN
  IF (SELECT rfq_id FROM Quotation WHERE quotation_id=NEW.quotation_id) <>
    (SELECT rfq_id FROM RFQ_Item WHERE rfq_item_id=NEW.rfq_item_id) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Quotation item must belong to the same RFQ';
```

```
    END IF;
END$$
CREATE TRIGGER trg_qi_same_rfq_before_update BEFORE UPDATE ON Quotation_Item FOR EACH ROW
BEGIN
    IF (SELECT rfq_id FROM Quotation WHERE quotation_id=NEW.quotation_id) <>
        (SELECT rfq_id FROM RFQ_Item WHERE rfq_item_id=NEW.rfq_item_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Quotation item must belong to the same RFQ';
    END IF;
END$$
CREATE TRIGGER trg_poi_same_quote_before_insert BEFORE INSERT ON PO_Item FOR EACH ROW
BEGIN
    IF (SELECT quotation_id FROM PurchaseOrder WHERE po_id=NEW.po_id) <>
        (SELECT quotation_id FROM Quotation_Item WHERE quotation_item_id=NEW.quotation_item_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PO item must belong to PO quotation';
    END IF;
END$$
CREATE TRIGGER trg_poi_same_quote_before_update BEFORE UPDATE ON PO_Item FOR EACH ROW
BEGIN
    IF (SELECT quotation_id FROM PurchaseOrder WHERE po_id=NEW.po_id) <>
        (SELECT quotation_id FROM Quotation_Item WHERE quotation_item_id=NEW.quotation_item_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PO item must belong to PO quotation';
    END IF;
END$$
CREATE TRIGGER trg_pm_role_before_insert BEFORE INSERT ON ProjectMember FOR EACH ROW
```

```
BEGIN
  IF (SELECT role FROM `User` WHERE user_id=NEW.user_id) <> 'PM' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Project member must have PM role';
  END IF;
END$$
CREATE TRIGGER trg_pm_role_before_update BEFORE UPDATE ON ProjectMember FOR EACH ROW
BEGIN
  IF (SELECT role FROM `User` WHERE user_id=NEW.user_id) <> 'PM' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Project member must have PM role';
  END IF;
END$$
CREATE TRIGGER trg_rfq_owner_before_insert BEFORE INSERT ON RFQ FOR EACH ROW
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM Project p JOIN `User` u ON u.user_id=NEW.created_by
    WHERE p.project_id=NEW.project_id AND p.owner_user_id=NEW.created_by AND u.role='GC'
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='RFQ creator must own project as GC';
  END IF;
END$$
CREATE TRIGGER trg_rfq_owner_before_update BEFORE UPDATE ON RFQ FOR EACH ROW
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM Project p JOIN `User` u ON u.user_id=NEW.created_by
    WHERE p.project_id=NEW.project_id AND p.owner_user_id=NEW.created_by AND u.role='GC'
```

```
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='RFQ creator must own project as GC';
  END IF;
END$$
CREATE TRIGGER trg_po_owner_before_insert BEFORE INSERT ON PurchaseOrder FOR EACH ROW
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM Quotation q JOIN RFQ r ON r.rfq_id=q.rfq_id
    JOIN Project p ON p.project_id=r.project_id
    JOIN `User` u ON u.user_id=NEW.created_by
    WHERE q.quotation_id=NEW.quotation_id AND q.status='accepted'
      AND p.owner_user_id=NEW.created_by AND u.role='GC'
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PO requires accepted quote and project-owning GC';
  END IF;
END$$
CREATE TRIGGER trg_po_owner_before_update BEFORE UPDATE ON PurchaseOrder FOR EACH ROW
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM Quotation q JOIN RFQ r ON r.rfq_id=q.rfq_id
    JOIN Project p ON p.project_id=r.project_id
    JOIN `User` u ON u.user_id=NEW.created_by
    WHERE q.quotation_id=NEW.quotation_id AND q.status='accepted'
      AND p.owner_user_id=NEW.created_by AND u.role='GC'
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PO requires accepted quote and project-owning GC';
  END IF;
```

```
END$$
CREATE TRIGGER trg_poi_quantity_before_insert BEFORE INSERT ON PO_Item FOR EACH ROW
BEGIN
  IF NEW.quantity > (SELECT quantity FROM Quotation_Item WHERE quotation_item_id=NEW.quotation_item_id) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PO quantity exceeds quoted quantity';
  END IF;
END$$
CREATE TRIGGER trg_poi_quantity_before_update BEFORE UPDATE ON PO_Item FOR EACH ROW
BEGIN
  IF NEW.quantity > (SELECT quantity FROM Quotation_Item WHERE quotation_item_id=NEW.quotation_item_id) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='PO quantity exceeds quoted quantity';
  END IF;
END$$
CREATE TRIGGER trg_user_role_before_update BEFORE UPDATE ON `User` FOR EACH ROW
BEGIN
  IF NEW.role <> OLD.role AND (
    EXISTS (SELECT 1 FROM Project WHERE owner_user_id=OLD.user_id)
    OR EXISTS (SELECT 1 FROM ProjectMember WHERE user_id=OLD.user_id)
    OR EXISTS (SELECT 1 FROM Supplier WHERE user_id=OLD.user_id)
    OR EXISTS (SELECT 1 FROM RFQ WHERE created_by=OLD.user_id)
    OR EXISTS (SELECT 1 FROM PurchaseOrder WHERE created_by=OLD.user_id)
  ) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Cannot change role while role-dependent records exist';
  END IF;
END$$
```

```
DELIMITER ;
```

The triggers provide an additional layer of data integrity by validating relationships and conditions that cannot be enforced through standard table constraints alone. For example, project ownership is restricted to users with the General Contractor role, while project membership requires the Project Manager role.

Other triggers ensure that quotation items belong to the correct RFQ, purchase order items correspond to the selected quotation, and ordered quantities do not exceed quoted quantities. An additional trigger prevents user role changes when existing records depend on the original role.

```
mysql> SELECT TRIGGER_NAME, EVENT_MANIPULATION, EVENT_OBJECT_TABLE
    -> FROM information_schema.TRIGGERS
    -> WHERE TRIGGER_SCHEMA = 'SUPPLIX_A3_SUBMISSION'
    -> ORDER BY EVENT_OBJECT_TABLE, TRIGGER_NAME;
+-----------------------------------+--------------------+--------------------+
| TRIGGER_NAME                      | EVENT_MANIPULATION | EVENT_OBJECT_TABLE |
+-----------------------------------+--------------------+--------------------+
| trg_poi_quantity_before_insert    | INSERT             | PO_Item            |
| trg_poi_quantity_before_update    | UPDATE             | PO_Item            |
| trg_poi_same_quote_before_insert  | INSERT             | PO_Item            |
| trg_poi_same_quote_before_update  | UPDATE             | PO_Item            |
| trg_project_gc_before_insert      | INSERT             | Project            |
| trg_project_gc_before_update      | UPDATE             | Project            |
| trg_pm_role_before_insert         | INSERT             | ProjectMember      |
| trg_pm_role_before_update         | UPDATE             | ProjectMember      |
| trg_po_owner_before_insert        | INSERT             | PurchaseOrder      |
| trg_po_owner_before_update        | UPDATE             | PurchaseOrder      |
| trg_qi_same_rfq_before_insert     | INSERT             | Quotation_Item     |
| trg_qi_same_rfq_before_update     | UPDATE             | Quotation_Item     |
| trg_rfq_owner_before_insert       | INSERT             | RFQ                |
| trg_rfq_owner_before_update       | UPDATE             | RFQ                |
| trg_supplier_role_before_insert   | INSERT             | Supplier           |
| trg_supplier_role_before_update   | UPDATE             | Supplier           |
| trg_user_role_before_update       | UPDATE             | User               |
+-----------------------------------+--------------------+--------------------+
17 rows in set (0.005 sec)
```

1.4. Create HistoricalTransaction table

To store historical material transaction records imported from the source dataset without treating them as purchase orders generated through the SUPPLIX application.

```
-- STEP 2: Historical transaction table (additional to 15 MVP tables)
-- A source transaction is not assumed to be an app PurchaseOrder.
CREATE TABLE HistoricalTransaction (
   transaction_id VARCHAR(20) PRIMARY KEY,
```

```
    supplier_id VARCHAR(20) NOT NULL,
    material_id VARCHAR(20) NOT NULL,
    quantity DECIMAL(18,4) NOT NULL,
    price_per_unit DECIMAL(18,4) NOT NULL,
    discount_rate DECIMAL(9,4) NOT NULL,
    reported_total DECIMAL(20,4),
    calculated_total DECIMAL(20,4) GENERATED ALWAYS AS
        (ROUND(quantity * price_per_unit * (1 - discount_rate / 100), 4)) STORED,
    transaction_date DATE,
    notes TEXT,
    CONSTRAINT fk_ht_supplier FOREIGN KEY (supplier_id) REFERENCES
Supplier(supplier_id),
    CONSTRAINT fk_ht_material FOREIGN KEY (material_id) REFERENCES
Material(material_id),
    CONSTRAINT chk_ht_quantity CHECK (quantity > 0),
    CONSTRAINT chk_ht_price CHECK (price_per_unit >= 0),
    CONSTRAINT chk_ht_discount CHECK (discount_rate BETWEEN 0 AND 100)
) ENGINE=InnoDB;
```

The HistoricalTransaction table stores validated historical transaction records. Each transaction is uniquely identified by transaction_id and linked to existing supplier and material records through foreign key constraints.

The quantity, price_per_unit, and discount_rate fields are used to calculate the transaction amount. The reported_total field preserves the value provided in the original dataset, while calculated_total is automatically generated using the validated numerical values.

CHECK constraints ensure that quantities are positive, unit prices are nonnegative, and discount rates remain between 0 and 100 percent. This structure supports reliable historical transaction analysis and enables comparisons between reported and calculated totals.

```
mysql> SHOW CREATE TABLE HistoricalTransaction\G
*************************** 1. row ***************************
       Table: HistoricalTransaction
Create Table: CREATE TABLE `HistoricalTransaction` (
  `transaction_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `supplier_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `material_id` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quantity` decimal(18,4) NOT NULL,
  `price_per_unit` decimal(18,4) NOT NULL,
  `discount_rate` decimal(9,4) NOT NULL,
  `reported_total` decimal(20,4) DEFAULT NULL,
  `calculated_total` decimal(20,4) GENERATED ALWAYS AS (round(((`quantity` * `price_per_unit`) * (1 - (`discount_rate` / 100))),4)) STORED,
  `transaction_date` date DEFAULT NULL,
  `notes` text COLLATE utf8mb4_unicode_ci,
  PRIMARY KEY (`transaction_id`),
  KEY `fk_ht_supplier` (`supplier_id`),
  KEY `fk_ht_material` (`material_id`),
  CONSTRAINT `fk_ht_material` FOREIGN KEY (`material_id`) REFERENCES `Material` (`material_id`),
  CONSTRAINT `fk_ht_supplier` FOREIGN KEY (`supplier_id`) REFERENCES `Supplier` (`supplier_id`),
  CONSTRAINT `chk_ht_discount` CHECK ((`discount_rate` between 0 and 100)),
  CONSTRAINT `chk_ht_price` CHECK ((`price_per_unit` >= 0)),
  CONSTRAINT `chk_ht_quantity` CHECK ((`quantity` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.004 sec)
```

1.5. Create staging tables

To temporarily store raw data imported from external TSV files before performing data validation, cleaning, and transformation.

Three staging tables were created to accommodate company information, material information, and historical transaction records. These tables preserve the imported source values, allowing data quality issues to be identified before loading records into the final database tables.

1.5.1. Create stg_materials table

To store raw material records imported from the materials.tsv source file.

```
-- STEP 3: Raw staging tables. Keep imported records intact.
CREATE TABLE stg_materials (
  staging_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  material_id VARCHAR(20),
  Item TEXT,
  Unit VARCHAR(50),
  PriceAvg VARCHAR(100),
  PriceStdev VARCHAR(100),
  NoSamples VARCHAR(100)
) ENGINE=InnoDB;
```

The stg_materials table stores the original material information, including material identifiers, names, units, average prices, price standard deviations, and sample counts.

An auto-incrementing staging_id is assigned to each imported record. The price-related fields are stored as VARCHAR values to preserve the original source data during the extraction and loading stage.

```
mysql> SHOW CREATE TABLE stg_materials\G
*************************** 1. row ***************************
       Table: stg_materials
Create Table: CREATE TABLE `stg_materials` (
  `staging_id` bigint NOT NULL AUTO_INCREMENT,
  `material_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `Item` text COLLATE utf8mb4_unicode_ci,
  `Unit` varchar(50) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `PriceAvg` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `PriceStdev` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `NoSamples` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  PRIMARY KEY (`staging_id`)
) ENGINE=InnoDB AUTO_INCREMENT=460 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

1.5.2. Create stg_companies table

To store raw supplier company information imported from the companies.tsv source file.

```
CREATE TABLE stg_companies (
  staging_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  company_id VARCHAR(20),
  name VARCHAR(200),
  description TEXT,
  address TEXT,
  phone VARCHAR(100),
  email VARCHAR(200),
  comments TEXT
) ENGINE=InnoDB;
```

The stg_companies table preserves the original company records before validation and transformation. It contains company identifiers, names, descriptions, addresses, contact information, and comments.

The staging_id field uniquely identifies each imported row, allowing invalid or duplicate records to be traced back to their original staging entries.

```
mysql> SHOW CREATE TABLE stg_companies\G
*************************** 1. row ***************************
       Table: stg_companies
Create Table: CREATE TABLE `stg_companies` (
  `staging_id` bigint NOT NULL AUTO_INCREMENT,
  `company_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `name` varchar(200) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `description` text COLLATE utf8mb4_unicode_ci,
  `address` text COLLATE utf8mb4_unicode_ci,
  `phone` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `email` varchar(200) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `comments` text COLLATE utf8mb4_unicode_ci,
  PRIMARY KEY (`staging_id`)
) ENGINE=InnoDB AUTO_INCREMENT=511 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.003 sec)
```

1.5.3. Create stg_transactions table

To store raw historical transaction records imported from the transactions.tsv source file.

```
CREATE TABLE stg_transactions (
  staging_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  transaction_id VARCHAR(20),
  company_id VARCHAR(20),
  material_id VARCHAR(20),
  quantity VARCHAR(100),
  price_per_unit VARCHAR(100),
  discount_rate VARCHAR(100),
  total_price VARCHAR(100),
  transaction_date VARCHAR(100),
  notes TEXT
) ENGINE=InnoDB;
```

The stg_transactions table stores the original transaction data before validation. It includes transaction identifiers, company and material references, quantities, prices, discount rates, reported totals, dates, and notes.

Numeric and date-related fields are initially stored as VARCHAR values to preserve raw inputs and allow validation before conversion into their final data types. The staging_id field enables individual source records to be tracked throughout the ETL process.

```
mysql> SHOW CREATE TABLE stg_transactions\G
*************************** 1. row ***************************
       Table: stg_transactions
Create Table: CREATE TABLE `stg_transactions` (
  `staging_id` bigint NOT NULL AUTO_INCREMENT,
  `transaction_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `company_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `material_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `quantity` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `price_per_unit` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `discount_rate` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `total_price` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `transaction_date` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `notes` text COLLATE utf8mb4_unicode_ci,
  PRIMARY KEY (`staging_id`)
) ENGINE=InnoDB AUTO_INCREMENT=51970 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

2. DATA EXTRACTION AND LOADING

The ETL process begins by extracting data from three tab-separated value (TSV) files: materials.tsv, companies.tsv, and transactions.tsv. The extracted records are loaded into their corresponding staging tables before validation and transformation.

2.1. Import material data

To import raw material records from the materials.tsv file into the stg_materials table.

```
-- STEP 4: Import files. Paths refer to MySQL server filesystem.
-- Explicit column lists are necessary because staging_id is auto-generated.
LOAD DATA INFILE '/var/lib/mysql-files/materials.tsv'
INTO TABLE stg_materials
CHARACTER SET utf8mb4
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES
(material_id, Item, Unit, PriceAvg, PriceStdev, NoSamples);
```

The LOAD DATA INFILE statement imports material records from the TSV file stored on the MySQL server. The FIELDS TERMINATED BY clause specifies tab-separated columns, while LINES TERMINATED BY defines the line-ending format.

The IGNORE 1 LINES option skips the header row. An explicit column list maps source fields to the staging table, while staging_id is generated automatically.

```
mysql> SELECT COUNT(*) AS total_material_records
    -> FROM stg_materials;
+------------------------+
| total_material_records |
+------------------------+
|                    458 |
+------------------------+
1 row in set (0.069 sec)
```

```
mysql> SELECT *
    -> FROM stg_materials
    -> LIMIT 5;
+------------+-------------+-----------------------------------------------------------------------------------------------------------------+------+
----+------------+-----------+
| staging_id | material_id | Item                                                                                                            | Unit |
Avg | PriceStdev | NoSamples |
+------------+-------------+-----------------------------------------------------------------------------------------------------------------+------+
----+------------+-----------+
|          1 | 85822412    | ordinary workers                                                                                                | Day  |
    | 276        | 193       |
|          2 | 14942603    | Product. Portland Cement. Type 1 Cement. Bags. Delivered to site.                                                | Bag  |
    | 19         | 172       |
|          3 | 03356886    | Equipment operator. Road roller                                                                                 | Hour |
    | 74         | 122       |
|          4 | 99529223    | Product: High-pressure concrete floor tiles (for Class C sidewalks, flat tiles). 60mm thick. Square.            | m♦   |
    | 85         | 97        |
|          5 | 36913810    | Products. Metal Materials. Wire                                                                                 | kg   |
    | 7          | 93        |
+------------+-------------+-----------------------------------------------------------------------------------------------------------------+------+
----+------------+-----------+
5 rows in set (0.002 sec)
```

2.2. Import company data

To import raw supplier company records from the companies.tsv file into the stg_companies staging table.

```
LOAD DATA INFILE '/var/lib/mysql-files/companies.tsv'
INTO TABLE stg_companies
CHARACTER SET utf8mb4
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES
(company_id, name, description, address, phone, email, comments);
```

The LOAD DATA INFILE statement imports supplier company information from the companies.tsv file into the stg_companies staging table. The file uses tab-separated columns, and the LINES TERMINATED BY clause specifies the line-ending format.

The IGNORE 1 LINES option skips the header row. The explicit column list maps the source fields to the corresponding staging table columns, while staging_id is automatically generated. The imported records are preserved for subsequent data validation and transformation.

```
mysql> SELECT COUNT(*) AS total_company_records
    -> FROM stg_companies;
+-----------------------+
| total_company_records |
+-----------------------+
|                   500 |
+-----------------------+
1 row in set (0.062 sec)
```

```
mysql> SELECT company_id, LEFT(name, 35) AS company_name, email
    -> FROM stg_companies
    -> LIMIT 5;
+------------+-------------------------------------+--------------------------------------+
| company_id | company_name                        | email                                |
+------------+-------------------------------------+--------------------------------------+
| 03028108   | Central Hardware                    | centralh@example.com                 |
| 19549147   | Premier Construction                | info@localtrade.net                  |
| 28001683   | Superior Foundations 99             | superior.foundations.99@mail.com     |
| 32120654   | Cornerstone Timber                  | cornerstone.timber82@constructive.co |
| 63786646   | Premier Prefabrication (dba Premier | proofing@supplyco.com                |
+------------+-------------------------------------+--------------------------------------+
5 rows in set (0.059 sec)
```

2.3. Import transaction data

To import raw historical transaction records from the transactions.tsv file into the stg_transactions staging table.

```
LOAD DATA INFILE '/var/lib/mysql-files/transactions.tsv'
INTO TABLE stg_transactions
CHARACTER SET utf8mb4
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES
(transaction_id, company_id, material_id, quantity, price_per_unit,
 discount_rate, total_price, transaction_date, notes);
```

The LOAD DATA INFILE statement imports historical transaction records from the transactions.tsv file into the stg_transactions staging table.

The source file contains transaction identifiers, company and material references, quantities, unit prices, discount rates, total prices, transaction dates, and notes.

The numeric and date-related values are initially stored as VARCHAR fields to preserve the original data before validation and transformation. This approach allows invalid or inconsistent records to be identified before loading them into the final HistoricalTransaction table.

```
mysql> SELECT COUNT(*) AS total_transaction_records
    -> FROM stg_transactions;
+--------------------------+
| total_transaction_records |
+--------------------------+
|                    51947 |
+--------------------------+
1 row in set (0.220 sec)
```

```
mysql> SELECT transaction_id, company_id, material_id, quantity
    -> FROM stg_transactions
    -> LIMIT 5;
+----------------+------------+-------------+----------+
| transaction_id | company_id | material_id | quantity |
+----------------+------------+-------------+----------+
| 74860229       | 08184297   | 86977837    | 291      |
| 13334298       | 19030448   | 68480503    | 55       |
| 85724846       | 58542299   | 56629388    | 325      |
| 04346803       | 51554249   | 72394227    | 65       |
| 97911208       | 75529600   | 87775215    | 254      |
+----------------+------------+-------------+----------+
5 rows in set (0.001 sec)
```

2.4. Verify imported data

To verify that the three source datasets have been successfully imported into their corresponding staging tables before performing data validation and transformation.

```
-- STEP 5: Quality review before transformation.
SELECT 'raw_companies' AS metric, COUNT(*) AS row_count FROM stg_companies
UNION ALL SELECT 'raw_materials', COUNT(*) FROM stg_materials
UNION ALL SELECT 'raw_transactions', COUNT(*) FROM stg_transactions;
```

The query uses COUNT(*) to calculate the number of records imported into each staging table. The UNION ALL operator combines the three counts into a single result, providing an overview of the imported datasets.

This verification step confirms that the raw data is available in the staging tables before the data quality assessment and transformation stages.

```
mysql> SELECT 'raw_companies' AS metric, COUNT(*) AS row_count FROM stg_companies
    -> UNION ALL SELECT 'raw_materials', COUNT(*) FROM stg_materials
    -> UNION ALL SELECT 'raw_transactions', COUNT(*) FROM stg_transactions;
+------------------+-----------+
| metric           | row_count |
+------------------+-----------+
| raw_companies    |       500 |
| raw_materials    |       458 |
| raw_transactions |     51947 |
+------------------+-----------+
3 rows in set (0.037 sec)
```

The verification results show that 500 company records, 458 material records, and 51,947 transaction records were successfully loaded into their respective staging tables, representing a total of 52,905 raw records. These results confirm the availability of the imported datasets for subsequent data validation, cleaning, and transformation.

3. DATA VALIDATION AND QUALITY ASSESSMENT

Before transforming and loading the imported data into the final application tables, a series of validation checks is performed to identify missing values, duplicate identifiers, invalid numerical values, and inconsistent references.

These checks help assess the quality of the source datasets and determine which records can be transferred into the final database and which records require rejection or further investigation.

3.1. Create ETL_RejectedMaster table

To create a separate table for recording invalid company and material records identified during the data validation process, while preserving the original data in the staging tables.

```
-- Duplicate IDs in master datasets must not be silently collapsed.
CREATE TABLE ETL_RejectedMaster (
  source_table VARCHAR(30) NOT NULL,
  staging_id BIGINT NOT NULL,
  source_id VARCHAR(20),
  reason VARCHAR(255) NOT NULL,
  PRIMARY KEY (source_table,staging_id)
) ENGINE=InnoDB;
```

The ETL_RejectedMaster table is created to store information about company and material records that fail the validation requirements. The source_table column identifies the original dataset, while staging_id and source_id allow rejected records to be traced back to their corresponding source entries.

The reason column records the cause of rejection, such as missing identifiers, missing required information, or duplicate IDs. The composite primary key (source_table, staging_id) ensures that each staging record can appear only once in the rejection table.

By maintaining rejected records separately, the ETL process preserves data traceability and prevents invalid master records from being transferred into the final application tables.

```
mysql> SHOW CREATE TABLE ETL_RejectedMaster\G
*************************** 1. row ***************************
       Table: ETL_RejectedMaster
Create Table: CREATE TABLE `ETL_RejectedMaster` (
  `source_table` varchar(30) COLLATE utf8mb4_unicode_ci NOT NULL,
  `staging_id` bigint NOT NULL,
  `source_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `reason` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  PRIMARY KEY (`source_table`,`staging_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.003 sec)
```

3.2. Validate company records

To identify company records with missing company IDs, missing company names, or duplicate company IDs before loading valid records into the Supplier table.

```
INSERT INTO ETL_RejectedMaster(source_table,staging_id,source_id,reason)
SELECT 'companies', c.staging_id, c.company_id,
      CASE WHEN c.company_id IS NULL OR TRIM(c.company_id) = '' THEN 'Missing company ID'
           WHEN c.name IS NULL OR TRIM(c.name) = '' THEN 'Missing company name'
           ELSE 'Duplicate company ID' END
FROM stg_companies c
JOIN (SELECT company_id, COUNT(*) AS n FROM stg_companies GROUP BY company_id) d
  ON c.company_id <=> d.company_id
WHERE c.company_id IS NULL OR TRIM(c.company_id) = ''
   OR c.name IS NULL OR TRIM(c.name) = '' OR d.n > 1;
```

The SQL statement validates company records stored in the stg_companies table by examining three potential data quality issues: missing company identifiers, missing company names, and duplicate company identifiers.

The CASE expression assigns a rejection reason based on the identified issue. The GROUP BY and COUNT(*) operations detect company IDs that appear more than once, while TRIM() identifies values containing only whitespace. The null-safe comparison operator (<=>) allows records with NULL identifiers to be matched during validation.

Records that fail these checks are inserted into ETL_RejectedMaster, where their original staging identifiers and rejection reasons are preserved for further investigation.

```
mysql> SELECT source_table, reason, COUNT(*) AS rejected_records
    -> FROM ETL_RejectedMaster
    -> WHERE source_table = 'companies'
    -> GROUP BY source_table, reason;
Empty set (0.011 sec)
```

3.3. Validate material records

To identify material records with missing material IDs, missing material names, missing units, or duplicate material IDs before transferring valid records into the Material table.

```
INSERT INTO ETL_RejectedMaster(source_table,staging_id,source_id,reason)
SELECT 'materials', m.staging_id, m.material_id,
      CASE WHEN m.material_id IS NULL OR TRIM(m.material_id) = '' THEN 'Missing material ID'
           WHEN m.Item IS NULL OR TRIM(m.Item) = '' THEN 'Missing material name'
           WHEN m.Unit IS NULL OR TRIM(m.Unit) = '' THEN 'Missing unit'
           ELSE 'Duplicate material ID' END
FROM stg_materials m
JOIN (SELECT material_id, COUNT(*) AS n FROM stg_materials GROUP BY material_id) d
 ON m.material_id <=> d.material_id
WHERE m.material_id IS NULL OR TRIM(m.material_id) = ''
  OR m.Item IS NULL OR TRIM(m.Item) = ''
  OR m.Unit IS NULL OR TRIM(m.Unit) = '' OR d.n > 1;
```

The material validation process examines the records in stg_materials to identify missing identifiers, missing material names, missing measurement units, and duplicate material IDs.

The CASE expression assigns a specific rejection reason to each invalid record. The GROUP BY and COUNT(*) operations identify duplicate material identifiers, while NULL and empty-string checks detect missing mandatory information.

Invalid material records are recorded in ETL_RejectedMaster rather than being transferred into the final Material table. This process helps maintain the integrity of the material master data while preserving rejected records for review.

3.4. Verify master data validation results

To summarize the number of rejected company and material records after the validation process.

```
mysql> SELECT
    ->     (SELECT COUNT(*) FROM stg_companies) AS raw_companies,
    ->     (SELECT COUNT(*) FROM stg_materials) AS raw_materials,
    ->     (SELECT COUNT(*) FROM ETL_RejectedMaster
    ->      WHERE source_table = 'companies') AS rejected_companies,
    ->     (SELECT COUNT(*) FROM ETL_RejectedMaster
    ->      WHERE source_table = 'materials') AS rejected_materials;
+---------------+---------------+--------------------+--------------------+
| raw_companies | raw_materials | rejected_companies | rejected_materials |
+---------------+---------------+--------------------+--------------------+
|           500 |           458 |                  0 |                  0 |
+---------------+---------------+--------------------+--------------------+
1 row in set (0.008 sec)
```

The verification query summarizes the total number of imported company and material records alongside the number of records rejected during validation.

The results provide an overview of the data quality assessment and help determine whether the source datasets contain records that fail the required validation conditions. These counts also support the subsequent loading of valid supplier and material records into the final application tables.

4. MASTER DATA TRANSFORMATION AND LOADING

After validating company and material records, the ETL process transfers valid source records into the application master tables. Records flagged in ETL_RejectedMaster are excluded, while the original staging data remains unchanged.

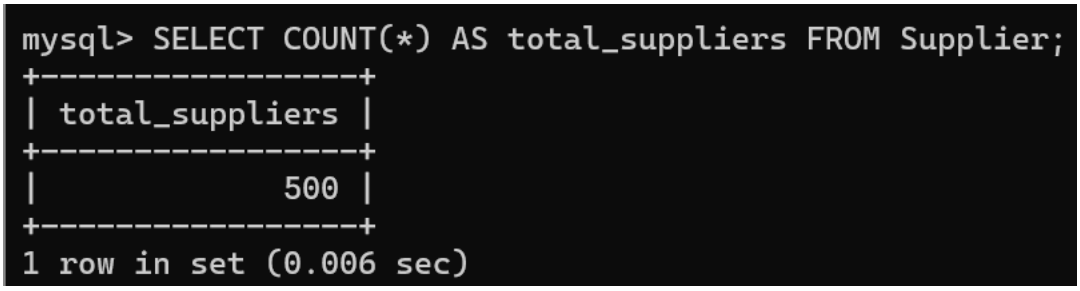
4.1. Load valid suppliers

To transfer valid company records from stg_companies into Supplier.

```
-- STEP 6: Load valid suppliers/materials. No dummy users or projects.
INSERT INTO Supplier (supplier_id,company_name,description,location,phone,email)
SELECT c.company_id,c.name,c.description,c.address,c.phone,c.email
```

```
FROM stg_companies c
LEFT JOIN ETL_RejectedMaster r
  ON r.source_table='companies' AND r.staging_id=c.staging_id
WHERE r.staging_id IS NULL;
```

The INSERT INTO ... SELECT statement maps company_id to supplier_id, name to company_name, and address to location. A LEFT JOIN with ETL_RejectedMaster identifies rejected company records; WHERE r.staging_id IS NULL ensures that only accepted records are inserted. The source files do not provide platform user accounts, so no dummy users are created.



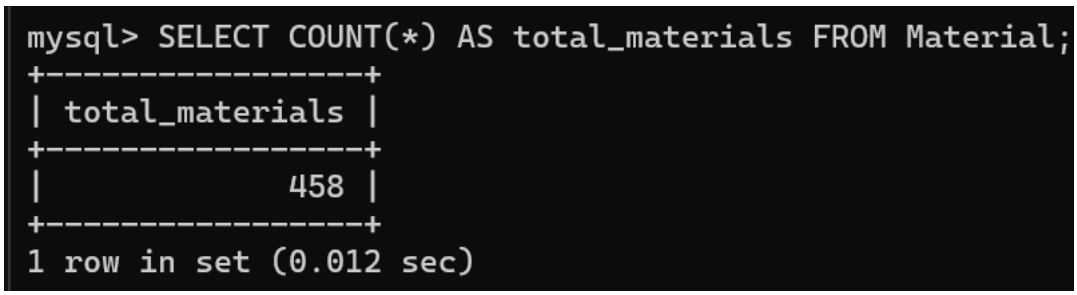
```
mysql> SELECT COUNT(*) AS total_suppliers FROM Supplier;
+-----------------+
| total_suppliers |
+-----------------+
|             500 |
+-----------------+
1 row in set (0.006 sec)
```

4.2. Load valid materials

To transfer validated material records from stg_materials into Material.

```
INSERT INTO Material (material_id,name,unit)
SELECT m.material_id,m.Item,m.Unit
FROM stg_materials m
LEFT JOIN ETL_RejectedMaster r
  ON r.source_table='materials' AND r.staging_id=m.staging_id
WHERE r.staging_id IS NULL;
```

The statement copies material identifiers, item names, and units to the final Material table. Rejected staging records are excluded using ETL_RejectedMaster. The source lacks category and specification values, so those optional attributes are not invented.



```
mysql> SELECT COUNT(*) AS total_materials FROM Material;
+-----------------+
| total_materials |
+-----------------+
|             458 |
+-----------------+
1 row in set (0.012 sec)
```

4.3. Preserve source semantics

```
-- Keep Day/Hour items: units alone do not prove a row is invalid.

-- No SupplierMaterial rows are invented: files contain no supplier-specific listings.
```

The ETL retains material units such as Day and Hour because these values alone do not establish invalidity. It also avoids fabricating supplier-specific listings in SupplierMaterial, since the source data does not contain the necessary supplier–material listing relationships.

5. HISTORICAL TRANSACTION VALIDATION AND LOADING

The historical transaction dataset is validated using a single decision process before records are either rejected or loaded. Raw values are retained in stg_transactions; temporary tables support safe conversion and classification without altering source records.

5.1. Create ETL_RejectedTransaction table

To preserve identifiers and reasons for transactions that fail validation.

```
-- STEP 7: Validate once, then use the same decision for reject and load.

-- This avoids inconsistent CASE/WHERE rules and unsafe casts of raw text.

CREATE TABLE ETL_RejectedTransaction (

  staging_id BIGINT PRIMARY KEY,

  transaction_id VARCHAR(20),

  reason VARCHAR(255) NOT NULL

) ENGINE=InnoDB;
```

The table uses staging_id as its primary key, ensuring that each rejected staging row is recorded once. The transaction_id and reason fields support traceability and investigation.

```
mysql> SHOW CREATE TABLE ETL_RejectedTransaction\G
*************************** 1. row ***************************
       Table: ETL_RejectedTransaction
Create Table: CREATE TABLE `ETL_RejectedTransaction` (
  `staging_id` bigint NOT NULL,
  `transaction_id` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `reason` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  PRIMARY KEY (`staging_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
1 row in set (0.002 sec)
```

5.2. Parse and validate raw transaction values

To check numeric precision, numerical formats, and date validity before type conversion.

```
-- Temporary review is recreated each time a fresh script execution begins.

-- Numeric conversion only occurs after strict format/precision validation.

CREATE TEMPORARY TABLE etl_tx_parsed AS

SELECT t.*,

  CASE WHEN TRIM(t.quantity) REGEXP '^[0-9]{1,14}([.][0-9]{1,4})?$'

       THEN CAST(TRIM(t.quantity) AS DECIMAL(18,4)) ELSE NULL END AS valid_quantity,

  CASE WHEN TRIM(t.price_per_unit) REGEXP '^[0-9]{1,14}([.][0-9]{1,4})?$'

      THEN CAST(TRIM(t.price_per_unit) AS DECIMAL(18,4)) ELSE NULL END AS valid_price,

  CASE WHEN TRIM(t.discount_rate) REGEXP '^[0-9]{1,5}([.][0-9]{1,4})?$'

       THEN CAST(TRIM(t.discount_rate) AS DECIMAL(9,4)) ELSE NULL END AS valid_discount,

  CASE WHEN t.total_price IS NULL OR TRIM(t.total_price) = '' THEN NULL

      WHEN TRIM(t.total_price) REGEXP '^[0-9]{1,16}([.][0-9]{1,4})?$'

       THEN CAST(TRIM(t.total_price) AS DECIMAL(20,4)) ELSE NULL END AS valid_reported_total,
```

```
 CASE WHEN t.transaction_date IS NULL OR TRIM(t.transaction_date) = '' THEN 1
     WHEN TRIM(t.transaction_date) REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$'
      AND SUBSTRING(t.transaction_date,1,4) <> '0000'
      AND SUBSTRING(t.transaction_date,6,2) BETWEEN '01' AND '12'
       AND CAST(SUBSTRING(t.transaction_date,9,2) AS UNSIGNED) BETWEEN 1 AND
       CASE WHEN SUBSTRING(t.transaction_date,6,2) IN ('04','06','09','11') THEN 30
           WHEN SUBSTRING(t.transaction_date,6,2) = '02' THEN
                   CASE WHEN MOD(CAST(SUBSTRING(t.transaction_date,1,4) AS UNSIGNED),400)=0
                            OR (MOD(CAST(SUBSTRING(t.transaction_date,1,4) AS UNSIGNED),4)=0
                           AND MOD(CAST(SUBSTRING(t.transaction_date,1,4) AS UNSIGNED),100)<>0)
                THEN 29 ELSE 28 END
           ELSE 31 END
     THEN 1 ELSE 0 END AS valid_date_flag
FROM stg_transactions t;
```

The temporary table etl_tx_parsed evaluates quantity, price_per_unit, discount_rate, total_price, and transaction_date. Regular expressions check whether numeric strings fit the destination DECIMAL precision before casting. The date validation checks the YYYY-MM-DD pattern, month lengths, and leap-year rules. Invalid inputs receive NULL parsed values or an invalid date flag rather than being silently coerced.

5.3. Classify transaction validation outcomes

To assign one rejection reason to each invalid transaction using a consistent set of rules.

```
CREATE TEMPORARY TABLE etl_tx_review AS
```

```
SELECT p.*,
  CASE
    WHEN p.transaction_id IS NULL OR TRIM(p.transaction_id) = '' THEN 'Missing transaction ID'
   WHEN d.n > 1 THEN 'Duplicate transaction ID'
   WHEN s.supplier_id IS NULL THEN 'Unknown or rejected supplier'
   WHEN m.material_id IS NULL THEN 'Unknown or rejected material'
   WHEN p.valid_quantity IS NULL OR p.valid_quantity <= 0 THEN 'Invalid quantity'
   WHEN p.valid_price IS NULL THEN 'Invalid price_per_unit'
     WHEN p.valid_discount IS NULL OR p.valid_discount > 100 THEN 'Invalid discount_rate'
   WHEN p.total_price IS NOT NULL AND TRIM(p.total_price) <> ''
       AND p.valid_reported_total IS NULL THEN 'Invalid reported total'
   WHEN p.valid_date_flag = 0 THEN 'Invalid transaction date'
   -- DECIMAL(20,4) allows at most 16 integer digits.
   WHEN (p.valid_quantity * p.valid_price * (1 - p.valid_discount / 100))
        > 9999999999999999.9999 THEN 'Calculated total exceeds DECIMAL(20,4)'
   ELSE NULL
  END AS rejection_reason
FROM etl_tx_parsed p
JOIN (SELECT transaction_id, COUNT(*) AS n
     FROM stg_transactions GROUP BY transaction_id) d
  ON p.transaction_id <=> d.transaction_id
LEFT JOIN Supplier s ON s.supplier_id = p.company_id
```

```
LEFT JOIN Material m ON m.material_id = p.material_id;
```

The CASE expression evaluates missing or duplicate transaction IDs, unknown suppliers, unknown materials, invalid quantities, invalid prices, invalid discounts, invalid reported totals, invalid dates, and totals exceeding the supported DECIMAL range. Its first matching condition determines the rejection_reason. Joining Supplier and Material checks whether referenced master records exist.

5.4. Store rejected transaction records

To retain transactions classified as invalid, including their reason for rejection.

```
INSERT INTO ETL_RejectedTransaction (staging_id, transaction_id, reason)

SELECT staging_id, transaction_id, rejection_reason

FROM etl_tx_review

WHERE rejection_reason IS NOT NULL;
```

Only rows with rejection_reason IS NOT NULL are inserted into ETL_RejectedTransaction. The same review decision will be used for loading accepted records, preventing inconsistencies between rejection and acceptance logic.

```
mysql> SELECT COUNT(*) AS rejected_transactions FROM ETL_RejectedTransaction;
+----------------------+
| rejected_transactions |
+----------------------+
|                 5112 |
+----------------------+
1 row in set (0.021 sec)
```

5.5. Load accepted historical transactions

To load valid source transactions without treating them as application-generated purchase orders.

```
-- Historical source transactions are not application PurchaseOrders.

-- The generated calculated_total uses the validated decimal inputs.

INSERT INTO HistoricalTransaction
```

```
  (transaction_id, supplier_id, material_id, quantity, price_per_unit,
   discount_rate, reported_total, transaction_date, notes)
SELECT transaction_id, company_id, material_id,
       valid_quantity, valid_price, valid_discount, valid_reported_total,
      CASE WHEN transaction_date IS NULL OR TRIM(transaction_date) = '' THEN NULL
           ELSE CAST(TRIM(transaction_date) AS DATE) END,
       notes
FROM etl_tx_review
WHERE rejection_reason IS NULL;
```

The INSERT INTO ... SELECT statement loads only rows with rejection_reason IS NULL. It uses validated numeric values, retains the source reported_total, and converts a validated date or stores NULL when no date was provided. calculated_total is generated automatically by HistoricalTransaction and is not inserted directly.



```
mysql> SELECT COUNT(*) AS accepted_transactions FROM HistoricalTransaction;
+----------------------+
| accepted_transactions |
+----------------------+
|                46835 |
+----------------------+
1 row in set (0.066 sec)
```

5.6. Remove temporary review tables

To release temporary intermediate ETL data after rejected and accepted records have been saved.

```
DROP TEMPORARY TABLE etl_tx_review;
DROP TEMPORARY TABLE etl_tx_parsed;
```

Both temporary tables are dropped after processing. Their removal does not delete source staging data, accepted historical transactions, or the persistent rejection records. These tables were session-specific; do not rerun their DROP commands simply to capture screenshots.

6. FINAL VERIFICATION AND ETL RECONCILIATION

The final stage verifies the loaded master data, accepted transactions, rejected records, financial calculations, referential summaries, and the completeness of the database structure.

6.1. Verify final table record counts

To confirm how many valid and rejected records were stored after ETL.

```
-- STEP 8: Verification

SELECT 'Suppliers' AS table_name, COUNT(*) AS row_count FROM Supplier

UNION ALL SELECT 'Materials', COUNT(*) FROM Material

UNION ALL SELECT 'HistoricalTransactions', COUNT(*) FROM HistoricalTransaction

UNION ALL SELECT 'RejectedMasterRows', COUNT(*) FROM ETL_RejectedMaster

UNION ALL SELECT 'RejectedTransactionRows', COUNT(*) FROM ETL_RejectedTransaction;
```

The UNION ALL query combines record counts for Supplier, Material, HistoricalTransaction, ETL_RejectedMaster, and ETL_RejectedTransaction. The observed totals are 500 suppliers, 458 materials, 46,835 accepted historical transactions, 0 rejected master records, and 5,112 rejected transactions.

```
mysql> SELECT 'Suppliers' AS table_name, COUNT(*) AS row_count FROM Supplier
    -> UNION ALL SELECT 'Materials', COUNT(*) FROM Material
    -> UNION ALL SELECT 'HistoricalTransactions', COUNT(*) FROM HistoricalTransaction
    -> UNION ALL SELECT 'RejectedMasterRows', COUNT(*) FROM ETL_RejectedMaster
    -> UNION ALL SELECT 'RejectedTransactionRows', COUNT(*) FROM ETL_RejectedTransaction;
+------------------------+-----------+
| table_name             | row_count |
+------------------------+-----------+
| Suppliers              |       500 |
| Materials              |       458 |
| HistoricalTransactions |     46835 |
| RejectedMasterRows     |         0 |
| RejectedTransactionRows |      5112 |
+------------------------+-----------+
5 rows in set (0.014 sec)
```

6.2. Review transaction rejection reasons

To identify the principal causes of transaction rejection.

```
SELECT reason, COUNT(*) AS rejected_rows

FROM ETL_RejectedTransaction GROUP BY reason ORDER BY rejected_rows DESC;
```

Grouping by reason summarizes rejected records by validation outcome. The observed rejection counts are 2,615 unknown or rejected suppliers and 2,497 unknown or rejected materials, totaling 5,112 rejected transactions.

```
mysql> SELECT reason, COUNT(*) AS rejected_rows
    -> FROM ETL_RejectedTransaction GROUP BY reason ORDER BY rejected_rows DESC;
+------------------------------+---------------+
| reason                       | rejected_rows |
+------------------------------+---------------+
| Unknown or rejected supplier |          2615 |
| Unknown or rejected material |          2497 |
+------------------------------+---------------+
2 rows in set (0.005 sec)
```

6.3. Check reported and calculated totals

To measure how many accepted transactions have a difference exceeding one unit between source-reported and computed totals.

```
SELECT COUNT(*) AS transactions_with_total_difference_over_1

FROM HistoricalTransaction

WHERE reported_total IS NOT NULL AND ABS(reported_total-calculated_total)>1;
```

The query compares reported_total and calculated_total for accepted records where the source total is present. A difference greater than one is flagged for review. The actual result should be copied from MySQL; it must not be assumed to be zero.

```
mysql> SELECT COUNT(*) AS transactions_with_total_difference_over_1
    -> FROM HistoricalTransaction
    -> WHERE reported_total IS NOT NULL AND ABS(reported_total-calculated_total)>1;
+-----------------------------------------+
| transactions_with_total_difference_over_1 |
+-----------------------------------------+
|                                   17520 |
+-----------------------------------------+
1 row in set (0.016 sec)
```

6.4. Review suppliers by calculated spending

To inspect the five suppliers with the highest calculated historical transaction value.

```
SELECT s.supplier_id,s.company_name,COUNT(*) AS total_transactions,
    ROUND(SUM(h.calculated_total),2) AS calculated_spend
FROM HistoricalTransaction h JOIN Supplier s ON h.supplier_id=s.supplier_id
GROUP BY s.supplier_id,s.company_name ORDER BY calculated_spend DESC LIMIT 5;
```

The query joins accepted historical transactions with supplier profiles, counts their transactions, and ranks suppliers by summed calculated_total. This provides a useful post-load sanity check without modifying any records.

```
mysql> SELECT s.supplier_id,s.company_name,COUNT(*) AS total_transactions,
    ->        ROUND(SUM(h.calculated_total),2) AS calculated_spend
    -> FROM HistoricalTransaction h JOIN Supplier s ON h.supplier_id=s.supplier_id
    -> GROUP BY s.supplier_id,s.company_name ORDER BY calculated_spend DESC LIMIT 5;
+-------------+------------------------------+--------------------+------------------+
| supplier_id | company_name                 | total_transactions | calculated_spend |
+-------------+------------------------------+--------------------+------------------+
| 91265491    | Riverbend Construction       |                109 |     152324796.00 |
| 64706568    | Riverbend Supply             |                104 |     114019624.00 |
| 12410582    | Horizon Logistics Enterprises |                 98 |     106944644.00 |
| 83347386    | Brown & Masonry              |                102 |     105749016.00 |
| 25045213    | Millennium Timber Enterprises |                 98 |     105619927.00 |
+-------------+------------------------------+--------------------+------------------+
5 rows in set (0.360 sec)
```

6.5. Review frequently transacted materials

To inspect the five most frequently transacted materials.

```
SELECT m.material_id,m.name,COUNT(*) AS times_transacted,
    ROUND(SUM(h.calculated_total),2) AS calculated_revenue
FROM HistoricalTransaction h JOIN Material m ON h.material_id=m.material_id
GROUP BY m.material_id,m.name ORDER BY times_transacted DESC LIMIT 5;
```

The query joins HistoricalTransaction to Material and summarizes transaction frequency and calculated value by material. The output helps check whether the imported records support meaningful material-level analysis.

```
mysql> SELECT m.material_id,m.name,COUNT(*) AS times_transacted,
    ->        ROUND(SUM(h.calculated_total),2) AS calculated_revenue
    -> FROM HistoricalTransaction h JOIN Material m ON h.material_id=m.material_id
    -> GROUP BY m.material_id,m.name ORDER BY times_transacted DESC LIMIT 5;
+-------------+-----------------------------------------------------------------------------------------------------------+--------------
---+--------------------+
| material_id | name                                                                                                      | times_transact
ed | calculated_revenue |
+-------------+-----------------------------------------------------------------------------------------------------------+--------------
---+--------------------+
| 81734598    | engineer                                                                                                  |            1
30 |       116472265.00 |
| 80801586    | Product. Ready-mix concrete material cost. 140kgf/cm♦. Type 1 cement. Delivery to construction site. |            1
28 |        65780940.00 |
| 70166708    | Product. 600V polyvinyl chloride insulated wire (PVC). Stranded wire. 5.5mm2                              |            1
28 |         2954986.00 |
| 12201654    | 1.2M3. Wheeled                                                                                            |            1
24 |        33286195.00 |
| 56851760    | Product. Planting. Added soil. Sandy loam                                                                 |            1
24 |        11555832.00 |
+-------------+-----------------------------------------------------------------------------------------------------------+--------------
---+--------------------+
5 rows in set (0.137 sec)
```

6.6. Reconcile raw, accepted, and rejected transactions

To verify that every source transaction is accounted for in either the accepted or rejected dataset.

```
SELECT (SELECT COUNT(*) FROM stg_transactions) AS raw_rows,
    (SELECT COUNT(*) FROM HistoricalTransaction) AS accepted_rows,
    (SELECT COUNT(*) FROM ETL_RejectedTransaction) AS rejected_rows,
    (SELECT COUNT(*) FROM stg_transactions)
     - (SELECT COUNT(*) FROM HistoricalTransaction)
     - (SELECT COUNT(*) FROM ETL_RejectedTransaction) AS difference;
```

The reconciliation computes raw_rows minus accepted_rows minus rejected_rows. With 51,947 raw transactions, 46,835 accepted transactions, and 5,112 rejected transactions, the difference is 0. This confirms that no transaction row is unaccounted for in the final classification.

```
mysql> SELECT (SELECT COUNT(*) FROM stg_transactions) AS raw_rows,
    ->        (SELECT COUNT(*) FROM HistoricalTransaction) AS accepted_rows,
    ->        (SELECT COUNT(*) FROM ETL_RejectedTransaction) AS rejected_rows,
    ->        (SELECT COUNT(*) FROM stg_transactions)
    ->         - (SELECT COUNT(*) FROM HistoricalTransaction)
    ->         - (SELECT COUNT(*) FROM ETL_RejectedTransaction) AS difference;
+----------+---------------+---------------+------------+
| raw_rows | accepted_rows | rejected_rows | difference |
+----------+---------------+---------------+------------+
|    51947 |         46835 |          5112 |          0 |
+----------+---------------+---------------+------------+
1 row in set (0.040 sec)
```

6.7. Verify master data coverage

To compare imported source master records against the final application master tables.

```
SELECT (SELECT COUNT(*) FROM stg_companies) AS raw_companies,
    (SELECT COUNT(*) FROM Supplier) AS imported_suppliers,
    (SELECT COUNT(*) FROM stg_materials) AS raw_materials,
    (SELECT COUNT(*) FROM Material) AS imported_materials;
```

The query compares raw company and material counts with loaded supplier and material counts. Both datasets have full coverage: 500 raw companies versus 500 suppliers and 458 raw materials versus 458 materials. No artificial application accounts, projects, or supplier-specific listings are added.

```
mysql> SELECT (SELECT COUNT(*) FROM stg_companies) AS raw_companies,
    ->        (SELECT COUNT(*) FROM Supplier) AS imported_suppliers,
    ->        (SELECT COUNT(*) FROM stg_materials) AS raw_materials,
    ->        (SELECT COUNT(*) FROM Material) AS imported_materials;
+---------------+--------------------+---------------+--------------------+
| raw_companies | imported_suppliers | raw_materials | imported_materials |
+---------------+--------------------+---------------+--------------------+
|           500 |                500 |           458 |                458 |
+---------------+--------------------+---------------+--------------------+
1 row in set (0.006 sec)
```

6.8. Verify database table and trigger counts

To verify that the expected permanent schema and business-rule triggers exist.

```
SELECT COUNT(*) AS tables_created FROM information_schema.tables
WHERE table_schema = DATABASE() AND table_type = 'BASE TABLE';
SELECT COUNT(*) AS triggers_created FROM information_schema.triggers
WHERE trigger_schema = DATABASE();
```

The schema contains 21 permanent tables: 15 application tables, one historical transaction table, three staging tables, and two rejection tables. The database also contains 17 business-rule triggers. These counts verify structural completeness after ETL.

```
mysql> SELECT COUNT(*) AS tables_created FROM information_schema.tables
    -> WHERE table_schema = DATABASE() AND table_type = 'BASE TABLE';
+----------------+
| tables_created |
+----------------+
|             21 |
+----------------+
1 row in set (0.004 sec)

mysql> SELECT COUNT(*) AS triggers_created FROM information_schema.triggers
    -> WHERE trigger_schema = DATABASE();
+------------------+
| triggers_created |
+------------------+
|               17 |
+------------------+
1 row in set (0.001 sec)
```